

Anonymous Two-Factor Authentication in Distributed Systems: Certain Goals Are Beyond Attainment

Ding Wang, *Student Member, IEEE*, Debiao He, Ping Wang, and Chao-Hsien Chu, *Senior Member, IEEE*

Abstract—Despite two decades of intensive research, it remains a challenge to design a practical anonymous two-factor authentication scheme, for the designers are confronted with an impressive list of security requirements (e.g., resistance to smart card loss attack) and desirable attributes (e.g., local password update). Numerous solutions have been proposed, yet most of them are shortly found either unable to satisfy some critical security requirements or short of a few important features. To overcome this unsatisfactory situation, researchers often work around it in hopes of a new proposal (but no one has succeeded so far), while paying little attention to the fundamental question: whether or not there are inherent limitations that prevent us from designing an “ideal” scheme that satisfies all the desirable goals? In this work, we aim to provide a definite answer to this question. We first revisit two foremost proposals, i.e. Tsai et al.’s scheme and Li’s scheme, revealing some subtleties and challenges in designing such schemes. Then, we systematically explore the inherent conflicts and unavoidable trade-offs among the design criteria. Our results indicate that, under the current widely accepted adversarial model, certain goals are beyond attainment. This also suggests a negative answer to the open problem left by Huang et al. in 2014. To the best of knowledge, the present study makes the first step towards understanding the underlying evaluation metric for anonymous two-factor authentication, which we believe will facilitate better design of anonymous two-factor protocols that offer acceptable trade-offs among usability, security and privacy.

Index Terms—Two-factor authentication, user anonymity, offline dictionary attack, de-synchronization attack, smart card loss attack

1 INTRODUCTION

PASSWORD authentication with smart card is one of the most convenient and effective two-factor authentication mechanisms in distributed systems, and it assures one communicating party of the authenticity of the corresponding party by acquisition of corroborative evidence. Although this technique has been widely deployed for various kinds of daily applications, such as e-banking, e-government and e-health, there are severe challenges regarding security [1], privacy [2] and usability [3] due to the open and complex nature of distributed systems, as well as the resource-constrained characteristics of mobile devices.

In 1999, Yang and Shieh [4] introduced the first smart-card-based password authentication scheme without a sensitive verification table stored on the server, which is a key advantage of two-factor schemes over common password-only schemes, for the latter (e.g., [5], [6]) have to maintain a sensitive password (or salted password) table on the server. Once this table is leaked, the entire system collapses. The feature of no password-related table on the server is highly appealing when considering the unending catastrophic

leakages of millions of user accounts in prominent service providers [7], [8] and the prevalence of zero-day attacks like the recent “Heartbleed” [9].

Since the seminal work of Yang and Shieh, there have been a great number of two-factor schemes suggested, and some notable ones include [10], [11], [12], [13]. In most of the previous two-factor schemes, user’s identity is transmitted in plain-text over public networks during the login process, which may leak the identity of the logging user once the login transcripts are eavesdropped, resulting in violation of user privacy and raising legal issues in some scenarios, e.g., electronic auditing or secret online-order placement. In many cases, an attacker may exploit the static user identity to link different login sessions together to trace user activities. For example, in e-commerce applications, once user activities are traced, the sensitive information such as shopping patterns, individual preferences, even age and gender, etc., can be learned and abused for marketing purposes, typically facilitating annoying advertisement flooding. What’s more, the disclosure of user identity and activities may also facilitate an unauthorized entity to trace the user’s login history and even current location [14]. To address such static-user-ID-related issues, a feasible approach is to adopt the “dynamic ID technique” [15]: the user’s real identity is concealed in session-variant pseudo-identities. And two-factor schemes employing this technique are known as “dynamic ID-based” or “anonymous” ones.

In 2004, Das et al. [15] introduced the first anonymous two-factor authentication scheme to preserve user privacy. Das’s work has been followed by a number of proposals [16], [17], [18] with various levels of security and diversity

- D. Wang and P. Wang are with the School of Electronics Engineering and Computer Science, Peking University, Beijing 100871, P.R. China. E-mail: wangdingg@mail.nankai.edu.cn, pwang@pku.edu.cn.
- D. He is with the School of Mathematics and Statistics, Wuhan University, Wuhan 430072, P.R. China. E-mail: hedeibiao@163.com.
- C.-H. Chu is with the College of Information Sciences and Technology, Pennsylvania State University, PA 16802. E-mail: chu@ist.psu.edu.

Manuscript received 26 Apr. 2014; revised 9 Aug. 2014; accepted 24 Aug. 2014. Date of publication 7 Sept. 2014; date of current version 10 July 2015. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below. Digital Object Identifier no. 10.1109/TDSC.2014.2355850

of attributes. A common feature of these schemes is that their security is based on the tamper-resistance assumption about smart cards, i.e., they simply assume that the security parameters stored in the smart card cannot be extracted. However, recent research results have demonstrated that common commercial smart cards shall no longer be considered to be fully tamper-proof: the secret information stored in the smart cards memory could be revealed by power analysis [19], reverse engineering techniques [20] or fault injection attacks [21]. As a consequence, such schemes based on the tamper-resistance assumption about the smart cards are susceptible to some types of attacks such as user impersonation attack and offline dictionary attack, once an adversary has breached the smart card. Therefore, it is more prudent and desirable to assume that once a smart-card is in the possession of an adversary, all the sensitive data stored in it are no longer secret. With this in mind, a number of anonymous schemes [11], [22], [23], [24] based on *non*-tamper-resistance assumption about the smart cards are put forward, and each is claimed to meet a self-imposed list of ambitious design goals.

1.1 Motivations

More often than not, the proponents assert the superiorities of their scheme, while (perhaps subconsciously) ignoring the features that their scheme fails to support, thus overlooking dimensions on which it fares poorly. This has contributed to a long-standing lack of progress on how best to evaluate proposals intended for practical applications. To address this imminent issue, in 2012, Madhusudhan and Mittal [25] developed a new set of design goals (including nine security requirements and ten desirable attributes) for fairly evaluating this type of schemes. Their set is a refinement of some previously proposed criteria sets (e.g., [10], [22], [26], [27]) and as far as we know, it is so far the most explicit, comprehensive and systematic criteria set for evaluating anonymous two-factor authentication schemes. In Madhusudhan-Mittal's work [25], it is concluded that all existing anonymous two-factor schemes are far from ideal and each has its own pros and cons, and it still remains an open problem as to how to design an ideal scheme that can satisfy all the criteria in their evaluation set.

To address this problem, a number of new solutions (e.g., [18], [23], [28]) have been suggested, followed by several cryptanalysis reports (e.g., [29], [30], [31]). This pattern of progress, once again, falls into the unsatisfactory "attack-fix-attack-fix" cycle (see [32, Fig. 1]). In this vicious cycle, protocol designers work around the above problem by presenting "improved" schemes but with not much confidence, while cryptanalysts respond to the above problem with concrete rebuttals to new proposals, yet no one pays attention to the underlying question: Whether a particular scheme is flawed due to improper design or whether there are some inherent limitations of this type of schemes that prevent us from designing "an ideal scheme"? Or equally, whether is it possible to construct an ideal scheme which satisfies all the design goals listed in [25]?

As far as we know, Huang et al.'s work [30] may be the closest to what we will discuss in the current paper, however, it mainly deals with security threats and challenges in

two-factor authentication and leaves over another interesting open problem as to "whether or not there exist secure smart-card-based password authentication protocols and the password-changing phase does not need any interaction with the server"?

Without these two fundamental questions addressed, we can only be kept stuck in the rut: lots of attempts are continually being contributed (and subsequently being defeated), yet little progress will be made.

1.2 Contributions

This study aims to provide definite answers to the above two questions. We first revisit the security and attribute provisions of two recent proposals, namely Tsai et al.'s scheme [29] and Li's scheme [33], and reveal some challenges and subtleties in designing anonymous two-factor schemes. These two schemes are among the foremost ones and claimed to be secure against various known attacks and to provide many admired features, yet as we will show, once again, they both fail to accommodate some important requirements. Remarkably, we figure out the fundamental flaw in their formal security proofs and highlight two practical threats, i.e. smart card loss attack and de-synchronization attack, the latter of which can be specially targeted at anonymous two-factor schemes.

Using these two representative schemes as case studies, we further investigate into the relationships among the criteria in Madhusudhan-Mittal's evaluation set [25], showing some inherent conflicts and unavoidable trade-offs in designing anonymous two-factor authentication schemes. Our results highly indicate that, under the current widely accepted adversarial model, certain goals are beyond attainment and therefore "an ideal scheme" is intrinsically out of reach. In particular, the revealed security-usability conflict also suggests a negative answer to the open problem left in [30]. To the best of knowledge, this study makes the first step toward exploring the inner relationships of evaluation criteria for anonymous two-factor authentication, which we believe will provide a much better understanding of how to design two-factor protocols that offer acceptable trade-offs among usability, security and privacy.

2 SYSTEM ARCHITECTURE, ADVERSARIAL MODEL AND EVALUATION CRITERIA

In this section, we elaborate on the system architecture, adversarial model and evaluation criteria. It is worth noting that, these three elements are key factors in determining whether a scheme has been evaluated systematically and fairly. There have been hundreds of papers dealing with smart-card-based password authentication quite recently (e.g., [11], [13], [16], [17], [34]), yet as far as we know, only a few ones [10], [13], [35] explicitly define these three elements (especially the later two elements) in their work, which may well explain why despite two decades of intensive research, there is still little consensus reached. Consequently, before stepping into the details of protocol specifications, we describe the system architecture, define the currently widely accepted adversarial model and introduce Madhusudhan-Mittal's criteria set [25].



Fig. 1. Smart-card-based password authentication.

2.1 System Architecture

In this work, as with [30], [32], we mainly focus on the most general case of smart-card-based password authentication (see Fig. 1), in which the participants involve a set of users and a single remote server. Typically, this kind of schemes consists of three basic phases, i.e. registration, authentication and password change, as well as some supplementary phases like eviction and revocation [24]. In the registration phase, a user submits some personal information to the server, and the server issues a smart card to the user. The smart card may contain some public and sensitive security parameters, which will be used later for the authentication. This phase is carried out only once unless the user re-registers. Upon accomplishment of the registration phase, the user is able to access the server in the authentication phase. This phase can be performed as many times as needed. What a truly two-factor scheme can ensure is that, only the user who possesses both a valid smart card and the corresponding password can be successfully verified by the server. In the password change phase, the user can change her password and update the information in the card either locally or by interacting with the server. To evict a malicious user and revoke a lost card, admired schemes may also provide additional phases such as eviction phase and revocation phase, respectively.

2.2 Adversarial Model

In the conventional password authenticated key exchange (PAKE) protocols (e.g., [5], [6]), the attacker $\mathcal{A}$ is generally assumed to be able to eavesdrop, block, alter or insert messages exchanged between the communicating parties, i.e., in full control of the communication channel. Besides, previous session key(s) may also be learnt by $\mathcal{A}$ due to a variety of reasons like improper erasure [36]. To capture the notion of forward secrecy, $\mathcal{A}$ may also be allowed to corrupt legitimate parties to learn long-term secrets.

Though these assumptions are reasonable for password-only authentication scenarios, it is inadequate for capturing practical threats in smart-card-based password authentication environments. As mentioned earlier, the secret data stored in the smart card, which was once believed to be free from breach, could be extracted by state-of-the-art side-channel attacks [19], [20], [21]. In addition, malicious card readers also contribute to the security failures of such schemes: a user's input password may be easily intercepted (key-logged) by a malicious card reader. It shall be noted that, as observed in [13] and further investigated in [32], $\mathcal{A}$ is unlikely to extract the secret information stored in the card while intercepting a victim's input password through malicious card readers, for the victim is on the scene and thus there is little chance for $\mathcal{A}$ to perform abnormal operations such as side-channel attacks. Note that, for the case

TABLE 1
Capabilities of the Adversary

C-01	The adversary $\mathcal{A}$ can enumerate offline all the items in the Cartesian product $\mathcal{D}_{id} * \mathcal{D}_{pw}$ within polynomial time, where $\mathcal{D}_{pw}$ and $\mathcal{D}_{id}$ denote the password space and the identity space, respectively.
C-02	The adversary $\mathcal{A}$ has the capability of somehow learning the victim's identity when evaluating security strength (but not privacy provisions) of the protocol.
C-1	The adversary $\mathcal{A}$ is in full control of the communication channel between the protocol participants.
C-2	The adversary $\mathcal{A}$ may either (i) learn the password of a legitimate user via malicious card reader, or (ii) extract the sensitive parameters in the card memory by side-channel attacks, but cannot achieve both.
C-3	The adversary $\mathcal{A}$ can learn the previous session key(s).
C-4	The adversary $\mathcal{A}$ is able to learn the server's long-time private key(s) only when evaluating the resistance to eventual failure of the system (e.g., forward secrecy).

where the smart card is sufficiently tamper-resistant such that the cost of attack is prohibitively high for an attacker, it is trivial to design an "ideal" scheme that satisfies all the goals listed in [25], and thus we omit it in this work.

Last but not least, it is practical to assume that a determined attacker can somehow know the victim's identity when having obtained the victim's card. First, user's identity is static and generally confined to a predefined structure, and thus it is of little cryptographic strength [37] and can be easily guessed. Second, it probably can be harvested from popular forums and other open resources. Third, users used to the idea of keeping passwords a secret would not normally be expecting to keep their identities a secret as well [38], e.g., writing their identities directly on the card. After all, $\mathcal{A}$ can learn more or less about the personal information of the card holder once she has gained access to the card. In a word, it is more reasonable to *do not* consider user identity as a surrogate extra password.

Here arises a subtlety to be explicated. This assumption about user identity is to emphasize that the security of a two-factor scheme shall not rely on the secrecy of user identities, and it is completely different from the assumption that $\mathcal{A}$ can determine a user's identity merely from the protocol transcripts. In other words, when dealing with *the security* of a scheme, it is more practical to regard user identity as a known value; however, when dealing with *the privacy provisions* of a scheme, the target user's identity is just what $\mathcal{A}$ endeavors to determine from the publicly available protocol transcripts.

The capabilities of the adversary $\mathcal{A}$ are summarized in Table 1. The adversarial model presented here is based on the models introduced in [30], [32], [35]. The only (and key) difference is that in our model, we for the first time explicitly define the adversary $\mathcal{A}$'s capabilities related to user identity from both the security perspective and the privacy perspective. This separation enables us to specifically deal with *anonymous* two-factor schemes.¹ Otherwise, some

1. In common two-factor schemes, user identities are transmitted in plain-text over the channel (i.e., without consideration of user privacy).

TABLE 2
Security Requirements

SR1	Resistance to DoS attack
SR2	Resistance to impersonation attack
SR3	Resistance to parallel session attack
SR4	Resistance to password guessing attack
SR5	Resistance to replay attack
SR6	Resistance to smart card loss attack
SR7	Resistance to stolen-verifier attack
SR8	Resistance to reflection attack
SR9	Resistance to insider attack

TABLE 3
Desirable Attributes

DA1	No password-related verifier table
DA2	Freely user password choice
DA3	No password reveal
DA4	Password dependent
DA5	Mutual authentication
DA6	Session key agreement
DA7	Forward secrecy
DA8	User anonymity
DA9	Smart card revocation
DA10	Efficiency for wrong password login

effective attacks specifically aiming at anonymous two-factor schemes can never be captured, such as the smart card loss attack presented in [38] and the offline password (and identity) guessing attack given in [39].

2.3 Evaluation Criteria

In 2012, Madhusudhan and Mittal [25] pointed out that earlier criteria sets, e.g. [26], [27], have ambiguities and redundancies, and thus they developed a new criteria set of nine security requirements (see Table 2) and ten desirable attributes (see Table 3) to evaluate the goodness of anonymous schemes. This criteria set is a refinement of earlier criteria sets, and interested readers are referred to [25] for the specific definition of each criterion. Here we only point out some subtleties, and the inner relationships among the criteria will be investigated in Section 5.

To be called “ideal”, a scheme should be able to satisfy all the nine security requirements and achieve all the ten desirable attributes. After putting forward the criteria set, Madhusudhan and Mittal [25] analyzed six recently proposed anonymous schemes and found none of existing ones can satisfy all the above nineteen criteria. Accordingly, they concluded that it remains an open problem to construct a scheme that can be considered ideal.

Madhusudhan-Mittal’s criteria set is superior to other proposed criteria mainly for the following two reasons: (1) The security requirements of their criteria set are based on the non-tamper-resistance assumption about the smart cards, which is desirable when taking into consideration the state-of-the-art side-channel attacks; (2) The separation of security requirements and desirable attributes makes the criteria set more concrete and facilitates protocol designers to establish a systematic approach for analyzing this type of schemes. Consequently, we prefer Madhusudhan-Mittal’s criteria set as a representative benchmark to other criteria sets (e.g. [10], [22], [26], [27]) in this study.

For a better comprehension, we address two subtleties (which are not made clear in [25]) related to the definitions of the above criteria. Firstly, security requirement SR6 relates to an adversary who has obtained the victim user’s smart card, while all the other security requirements (e.g., SR2 and SR4) are faced with an adversary who is without the victim user’s smart card. Secondly, in the context of remote user authentication, user anonymity (i.e., DA8) generally involves two aspects, i.e. identity protection and user un-traceability [2], [11]. Both are defined against the public (i.e., eavesdropping attackers) rather than the server, because the server has to first identify the

legitimacy of the user and then obtain the user’s real identity for accounting and/or billing purposes. This means that user anonymity is not a design goal when the server is compromised.

3 CRYPTANALYSIS OF TSAI ET AL.’S SCHEME

In 2013, Tsai et al. [29] showed some severe security pitfalls in Li et al.’s scheme [11] and proposed a new one. It is claimed that their new scheme achieves user un-traceability while being secure against general threats. In this work, however, we will demonstrate that, under their own assumptions, Tsai et al.’s scheme [29] actually cannot provide truly two-factor security, which is the most critical goal that any two-factor scheme shall attain. Moreover, as the result of overlooking an inherent usability-security trade-off, this scheme is subject to smart card loss attack.

3.1 Review of Tsai et al.’s Scheme

In this section, we briefly describe the two-factor scheme proposed by Tsai et al. [29] in 2013. Their scheme has two versions: one with fingerprint information involved and the other not. The biometric-involved version is essentially a three-factor scheme, yet Tsai et al. never discussed the issues and challenges incurred by introducing the third authentication factor (i.e., the biometric). Based on Huang et al.’s work [12], one can easily find that the three-factor version of Tsai et al.’s scheme is likely to be prone to the issues of biometric error-tolerance and non-trusted devices. What’s more, the non-biometric-involved version constitutes the basis of the biometric-involved version. Hence, we mainly focus on the two-factor version. Their scheme consists of five phases, namely, parameter generation, registration, pre-computation, login and password update. For ease of presentation, we employ some intuitive abbreviations and notations listed in Table 4.

3.1.1 Parameter Generation Phase

This scheme is based on Elliptic Curve Cryptosystem (ECC). First of all, the server S chooses an elliptic curve E_p over a finite field $\mathbb{F}_p$, a base point G with order n and its private key x . Then, S computes $P_S = x \times P$ as its public key, selects two hash functions $h(\cdot) : \{0, 1\}^* \rightarrow \{0, 1\}^k$ and $h_1(\cdot) : \{0, 1\}^* \rightarrow \{0, 1\}^{k'}$, where $|k| = |x|$ and $|k'| \geq |k|$, e.g., $|k| = |x| = 128$ bits, $|k'| = 256$ bits. In the end, S publishes security parameters $\{p, E_p, G, P_S, n, h(\cdot), h_1(\cdot)\}$.

TABLE 4
Notations and Abbreviations

Symbol	Description
U_i	i th user
S	remote server
ID_i	identity of user U_i
CID_i	dynamic identity of user U_i
PW_i	password of user U_i
x	the secret key of remote server S
p	a large prime number
O	the point at infinity
P	base point of the elliptic curve E_p where $n \cdot P = O$
$h(\cdot), h_1(\cdot)$	common collision free one-way hash functions
$\oplus$	the bitwise XOR operation
$\parallel$	the string concatenation operation
$\rightarrow$	a common communication channel
$\Rightarrow$	a secure communication channel

Remark 1. The original specification in [29] do not specify the length of k' , which we think is a key factor in conducting formal security reductions, especially in the random oracle model (ROM). Hence, we assume that $|k'| \geq |k|$ according to the security reductions in [29]. Moreover, the parameter $h_1(\cdot)$ is not published in the original specification. As the user needs $h_1(\cdot)$ to compute her authenticator in the login phase, $h_1(\cdot)$ should have also been published. Consequently, we deduce a typo has occurred here.

3.1.2 Registration Phase

As mentioned earlier, we hereafter only focus on the version with no biometric factor involved. Whenever U_i wants to register in S , the following operations are performed:

- 1) U_i selects her identity ID_i , password PW_i and a random number $b \in_R \mathbb{Z}_p^*$.
- 2) $U_i \Rightarrow S : \{ID_i, h(PW_i \parallel b)\}$.
- 3) On receiving the registration message from user U_i , the server S computes $V = h(ID_i \parallel x) \oplus h(PW_i \parallel b)$.
- 4) $S \Rightarrow U_i$: A security parameter V .
- 5) Upon receiving V , U_i keys b and V into the smart card.

3.1.3 Pre-Computation Phase

The smart card selects $N_{C1} \in_R \mathbb{Z}_p^*$, computes $e = N_{C1} \times P$ and $c = N_{C1} \times P_S$, and then stores $\{e, c, N_{C1}\}$ into its memory. This phase is executed only after the session key has been successfully established between U_i and S . In this way, the card is released from wasting time to do these computations in the next login.

3.1.4 Login Phase

When U_i wants to access S , the following steps proceed:

- 1) U_i retrieves $h(ID_i \parallel x)$ by computing $h(ID_i \parallel x) = V \oplus h(PW_i \parallel b)$, selects a random number N_{C2} , then calculates $C_1 = (ID_i \parallel (h(ID_i \parallel x) \oplus N_{C2})) \oplus h_1(c)$.
- 2) $U_i \rightarrow S : \{C_1, e\}$.

- 3) S retrieves ID_i and $h(ID_i \parallel x) \oplus N_{C2}$ by computing $ID_i \parallel (h(ID_i \parallel x) \oplus N_{C2}) = C_1 \oplus h_1(x \times e)$. Next, S gets N_{C2} by computing $N_{C2} = h(ID_i \parallel x) \oplus N_{C2} \oplus h(ID_i \parallel x)$. Then, S selects $N_S \in_R \mathbb{Z}_p^*$, computes $C_2 = N_S \times P$, $SK = h(e \parallel C_2 \parallel N_S \times e)$ and $C_3 = h(h(ID_i \parallel x) \parallel N_{C2} \parallel C_2 \parallel e \parallel SK)$.
- 4) $S \rightarrow U_i : \{C_2, C_3\}$.
- 5) On receiving the response from S , U_i computes $SK = h(e \parallel C_2 \parallel N_{C1} \times C_2) = h(e \parallel C_2 \parallel N_{C1} \times N_S \times P)$ and $C'_3 = h(h(ID_i \parallel x) \parallel N_{C2} \parallel C_2 \parallel e \parallel SK)$. Then, U_i checks whether the received C_3 equals C'_3 . If they are not equal, U_i rejects. Otherwise, U_i computes $C_4 = h(h(ID_i \parallel x) \parallel SK \parallel N_{C2} \parallel C_2 \parallel e)$.
- 6) $U_i \rightarrow S : \{C_4\}$.
- 7) S first computes $C'_4 = h(h(ID_i \parallel x) \parallel SK \parallel N_{C2} \parallel C_2 \parallel e)$ and then compares whether C'_4 equals the received C_4 . If it does not hold, U_i is rejected. Otherwise, S grants U_i 's request.

3.1.5 Password Change Phase

This phase is provided to allow users to change their passwords freely and locally. When U_i wants to change her password, she inserts her smart card into a terminal, keys her old password PW_i and the new one PW_i^{new} . Then, the card computes $V^{new} = V \oplus h(PW_i \parallel b) \oplus h(PW_i^{new} \parallel b) = h(ID_i \parallel x) \oplus h(PW_i^{new} \parallel b)$ and replaces V with V^{new} .

3.2 Cryptanalysis of Tsai et al.'s Scheme

Tsai et al. present five kinds of adversarial models and provide formal security proofs for their scheme under each model. It is not difficult to see that, their five adversarial models have been included into our model (see Section 2.2). Although their scheme exhibits many attractive properties over existing schemes, such as user un-traceability, high efficiency and formal security proofs, it is still far from an "ideal" anonymous two-factor protocol to be applicable for practical applications. In this section we will show that, it actually fails to resist smart card loss attack (i.e., SR6) under their own assumptions and is prone to smart card revocation problem (i.e., DA9). Now a paradoxical question arises: How can a protocol that was formally proven secure later be found insecure? We further explicate this paradox by showing that its security proofs are fallacious.

3.2.1 Smart Card Loss Attack I

What a two-factor protocol with resistance to smart card loss attack (i.e., SR6) can guarantee is that when user's smart card is lost (or stolen), there is no way for an adversary A to easily change the user's password, guess the password of the user using password guessing attacks, or impersonate the user to freely enjoy the services. What concerns us is the realistic possibilities that users lose their smart cards. Recent studies [40] on the usability of real-life two-factor systems have confirmed that, users do tend to leave their smart card unattended: 54 percent users have forgotten their smart cards in the card reader at least once during the study (i.e., a period of merely six weeks). Therefore, SR6 is a basic goal that any practical scheme shall attain. Unfortunately, the past research (e.g., [30], [32]) has proved that achieving this

TABLE 5
Computation Evaluation of Related Operations on Common PCs

Experimental Platform (common PCs)	ECC Point Multiplication $T_P(\text{ECC sect163r1 [44]})$	Modular Exponentiation $T_E(n = 512)$	Symmetric decryption $T_S(\text{AES-128})$	Hash operation $T_H(\text{SHA-1})$	Other lightweight operations(e.g.,XOR)
Intel T5870 2.00 GHz	1.226 ms	2.573 ms	2.049 μs	2.580 μs	0.011 μs
Intel E5500 2.80 GHz	0.617 ms	1.348 ms	0.572 μs	0.753 μs	0.009 μs
Intel i3-530 2.93 GHz	0.508 ms	1.169 ms	0.541 μs	0.693 μs	0.008 μs

goal is notoriously difficult. In the following, we show that, once more, Tsai et al.'s attempt ends in vain— $\mathcal{A}$ can obtain the user's password by an offline password guessing attack once the user's smart card is in the possession of $\mathcal{A}$.

Suppose an adversary $\mathcal{A}$ has somehow obtained (stolen or picked up) user U_i 's smart card for a relative long period of time (e.g., a few hours), and extracted the secret information $\{e, c, N_{C1}, V, b\}$ by using side-channel attacks [19], [20], [21] herself (or with recourse to professional labs). Then, $\mathcal{A}$ returns the breached card back to U_i without U_i 's awareness. Once user U_i uses the breached smart card to login, the attacker can intercept U_i 's login request $\{C_1, e, C_2, C_3\}$ and then obtains PW_i as follows:

- Step 1. Guesses the value of PW_i to be PW_i^* from the dictionary space $\mathcal{D}_{pw}$.
- Step 2. Computes $h(ID_i \| x)^* = V \oplus h(PW_i^* \| b)$, where V, b is extracted from U_i 's smart card.
- Step 3. Retrieves $h(ID_i \| x) \oplus N_{C2}$ by computing $ID_i \| (h(ID_i \| x) \oplus N_{C2}) = C_1 \oplus h(c)$, where c is extracted from U_i 's smart card and C_1 is eavesdropped.
- Step 4. Computes $N_{C2}^* = (h(ID_i \| x) \oplus N_{C2}) \oplus h(ID_i \| x)^*$, $SK = h(e \| C_2 \| N_{C1} \times C_2)$ and $C_3^* = h(h(ID_i \| x)^* \| N_{C2}^* \| C_2 \| e \| SK)$.
- Step 5. Verifies the correctness of PW_i^* by checking if the computed C_3^* is equal to the intercepted C_3 .
- Step 6. Repeats Step 1 ~ 5 of this procedure until the correct value of PW_i is found.

In the above attack, we have made two assumptions: (1) $\mathcal{A}$ can obtain and breach the victim's card; and (2) $\mathcal{A}$ can return the breached card *without detection*. The former assumption has been common in the literature (e.g., [10], [13], [41]) and it is also explicitly made in Tsai et al.'s work [29], while the latter has only recently raised attention and is indeed reasonable [30], [32].² Let $|\mathcal{D}_{pw}|$ denote the number of passwords in $\mathcal{D}_{pw}$. The time complexity of the above attacking procedure is $O(|\mathcal{D}_{pw}| * (T_P + 4T_H + T_X))$, where T_P is the running time for ECC point multiplication, T_H the running time for Hash function, T_X the running time for bitwise XOR operation. It is easy to see that, the time for $\mathcal{A}$ to recover U_i 's password is a linear function of $\mathcal{D}_{pw}$. And hence our attack is quite effective.

To gain a better grasp of the effectiveness of this attack, we further implement the related operations on common PCs and obtain the operation timings (see

Table 5), by using the publicly-available, rational arithmetic C/C++ library MIRACL [42]. In practice, due to the inherent limitations of human cognition, user passwords are often memorable short strings and hence the password space is very restricted, e.g., $|\mathcal{D}_{pw}| \leq 10^6$ [43], and it follows that $\mathcal{A}$ can complete the above attack in seconds on a common PC.

Our attack shows that once the smart-card factor is compromised, the corresponding password factor can be offline guessed and hence the entire system collapses. This indicates that Tsai et al.'s scheme is not a truly two-factor scheme, while the original scheme (i.e., Li et al.'s scheme [11]) does not suffer from this pitfall. The failure of Tsai et al.'s scheme is mainly attributed to the pre-computation phase, which facilitates $\mathcal{A}$ to obtain not only the long-term parameters but also the session-specific values (e.g., the parameter c). To eliminate this pitfall, an intuitive solution is to remove the pre-computation phase and compute e and c in the login phase, however this is at the cost of efficiency. We also note that, Li et al.'s scheme [11] employs a similar pre-computation technique and is free from the above attack, but it is at the price of de-synchronization attack (a kind of denial of service attack, see Section 4.2.2). Naturally, one may ask, whether there exists a secure anonymous two-factor scheme that employs the pre-computation technique? This question is very interesting yet out of the scope of this work, and we prompt it as an open problem.

3.2.2 Smart Card Loss Attack II

To support local user password update (i.e., DA2) like that of the schemes in [10], [17], [23], [24], the password change phase of Tsai et al.'s scheme is performed locally and does not need to interact with the remote server, which is in favor of user friendliness. However, this phase introduces an inherently insecure factor: there is no verification of the authenticity of the old password before the update of new password. If an attacker manages to gain temporary access to the smart card of legitimate user U_i (note that this is a quite realistic assumption as discussed in Section 3.2.1), she can easily change the password of user U_i as follows:

- Step 1. The attacker inserts U_i 's smart card into a card reader and initiates a password change request.
- Step 2. The attacker submits a random string R as U_i 's original password and a new string PW_i^{new} as the target-new password.
- Step 3. The smart card computes $V^{new} = V \oplus h(R \| b) \oplus h(PW_i^{new} \| b)$ and replaces V with V^{new} .

Once the value of V is updated, legitimate user U_i cannot login successfully even after getting her smart card back

2. In [30], Huang et al. cryptanalyze Juang et al.'s scheme and state that "an adversary $\mathcal{A}$ can calculate the session key if $\mathcal{A}$ extracts the information in the smart card before the log-in phase". This indicates they have implicitly made the assumption that $\mathcal{A}$ can return the breached card without detection and the victim will use the breached card to login.

because $V^{new} \oplus h(PW_i \| b) \neq h(ID_i \| x)$, and thus U_i 's subsequent login request will be denied by the server S during all the following login phases. This means denial of service attack can be launched easily once the card is lost, and thus this scheme fails to fulfill SR6.

Actually, this design flaw may also give rise to *another* quite practical and troublesome problem: if a legitimate user accidentally keys an incorrect value for the current (old) password in the password change process, the parameter V will be updated to an unpredictable (random) value. From now on, the smart card will become completely unusable unless the user re-registers with the server.

It is worth noting that, although these two vulnerabilities seem too basic to merit discussion, they are really practical and cannot be well conquered just with minor revisions. To eliminate them (i.e., achieving SR6) while preserving DA2, a verification of the authenticity of the original password before updating the value of V in the card memory is essential. And thus, besides V , some additional parameter(s) should be stored in the smart card. Note that this may introduce new vulnerabilities, such as offline guessing attack and impersonation attack. This subtlety has also been observed by Nam et al. [45], but unfortunately, they left it as an open problem. Most subsequent works either just overlook this issue [10], [16], [17], [22], [24] or choose not to provide local user password change [11], [33], [41], while the few rest [23], [28], [34] that are ambitious to both support local password change (i.e., DA2) and resist against the above smart card loss attack II (i.e., SR6) are all found prone to offline guessing attack [31], [39].

To gain more insights into this problem, here we give a concrete example. Suppose a supplementary parameter $A_i = h(ID_i \| h(PW_i))$ is stored in the smart card. Whenever U_i wants to update her password, first she must input her identity ID_i^* and password PW_i^* , then the smart card verifies whether $h(ID_i^* \| h(PW_i^*))$ is equal to the stored A_i . It is not difficult to see that $\mathcal{A}$ could exhaustively enumerate all the (ID_i, PW_i) pairs and determine the correct one in an offline manner once the parameter A_i has been obtained, which definitely leads to an offline guessing attack.

However, if the parameter A_i is computed as $A_i = h(\tilde{h}(ID_i) \oplus \tilde{h}(PW_i))$, it is not difficult to check that there exist $\frac{|\mathcal{D}_{id}| \cdot |\mathcal{D}_{pw}|}{2^8} \approx 2^{32}$ candidates of (ID, PW) pair to thwart $\mathcal{A}$ when $|\mathcal{D}_{id}| = |\mathcal{D}_{pw}| = 10^6$ [43], where $\tilde{h}(\cdot)$ is a special one-way hash function $\{0, 1\}^* \rightarrow \{0, 1, \dots, 255\}$, $|\mathcal{D}_{id}|$ and $|\mathcal{D}_{pw}|$ denote the size of the identity space and password space, respectively. Even if ID_i is also leaked to $\mathcal{A}$, there still exist $\frac{|\mathcal{D}_{pw}|}{2^8} \approx 2^{12}$ candidates of PW_i , each of which can only be excluded by an online guessing attack, while online guessing can be effectively thwarted [46]. In this way, $\mathcal{A}$ is prevented from obtaining the exactly correct PW_i and we call A_i computed through this new approach "a fuzzy verifier", which achieves the same effect with that of [35]. One may wonder what if U_i happens to input a wrong (ID_i^*, PW_i^*) pair such that $h(\tilde{h}(ID_i^*) \oplus \tilde{h}(PW_i^*)) = A_i$, while $(ID_i^*, PW_i^*) \neq (ID_i, PW_i)$? The reality is that this possibility is only $\frac{1}{2^{56}}$, which will be reduced to $\frac{1}{2^{56^2}}$ if we further require the user types her old/new passwords twice whenever changing password and if $\tilde{h}(\cdot)$ behaves like a random oracle.

An immediate "by-product" of this "fuzzy verifier" is that it can be used to provide timely wrong password detection when login (i.e., DA10).

Therefore, only radical changes in Tsai et al.'s scheme can completely eliminate the above pitfalls, and we conjecture that there is an unavoidable trade-off when fulfilling the criteria DA2, DA10 and SR6, which will be further discussed in Section 5.3. Employing "a fuzzy verifier" seems a good choice to deal with this problem, yet its practical effectiveness can only be testified by real-life password data sets. Fortunately, a number of recent catastrophic leaks of thousands of millions web accounts (e.g., Evernote [7] and LinkedIn [8]) have provided wonderful materials for this use, and it constitutes one of our future work.

3.2.3 Card Revocation Problem

In Tsai et al.'s scheme [29], to support the criterion DA10 (i.e., no password-related verification table stored on the server), there is no user-related (or card-related) data kept on the server at all. Of course, DA10 can be provided. But going too far is absolutely undesirable. As no card-related information stored in the server, there is no way for the server to tell apart a valid card from an invalid card, which means invalid (or expired) cards cannot be revoked.

What's more, Tsai et al.'s scheme is not easily repairable when the user re-registers with the server after the user finds that her smart card has been lost and/or the critical parameter $h(ID_i \| x) = V \oplus h(PW_i \| b)$ is somehow obtained by an adversary. As described in Section 3.2.1, there are non-negligible possibilities that $h(ID_i \| x)$ may be leaked to $\mathcal{A}$ once the user's card is lost. In this case, impersonation attacks cannot be prevented even if U_i finds that her card has been out of control and then re-registers with S . As the value of $h(ID_i \| x)$ is computed only with the contribution of U_i 's identifier ID_i and S 's permanent secret key x , S cannot update $h(ID_i \| x)$ for U_i unless ID_i or x can be modified to a new one. However, since x is generally related to all registered users rather than specifically used for U_i only, it is inefficient and virtually unrealistic if x is changed to restore the security of U_i only. On the other hand, it is impractical to change ID_i , which is unalterable in most scenarios and may be bound to U_i in many application systems.

3.2.4 Flaws in Tsai et al.'s Formal Security Proof

Generally speaking, a two-factor protocol achieving semantic security or the so-called "AKE security" [5], [35] under the non-tamper resistance assumption about the smart card (i.e., C-2i in Table 1 or the smart-card-lose case in [29]) can provide a basic level of security, such as resistance to impersonation attack and offline password guessing attack, even if the card has been lost and breached. Tsai et al.'s protocol is armed with a claimed proof of semantic security, yet as shown earlier, it cannot withstand offline password guessing attack once the smart-card factor is compromised.

Now a paradox arises: *How can a protocol that was proved secure later turn out to be insecure?* To address this interesting question, we scrutinize the reductionist security arguments of Tsai et al.'s protocol and manage to uncover the fundamental flaw in their reasoning of the proof.

The security model adopted by Tsai et al. [29] is based on the work of Xu et al. [41], while the latter is essentially a variant of the random oracle model introduced by Bellare et al. [5]. The general rationale that lies behind a proof of semantic security in ROM is: (1) Modelling any hash function as an oracle which outputs a random value for each new query and the same value for every identical query; (2) Supposing $\mathcal{A}$ can break the semantic security of the target protocol $\mathcal{P}$; (3) Exploiting $\mathcal{A}$ to build algorithms for each of the underlying cryptographic primitives (e.g., integer factoring assumption, computational Diffie-Hellman assumption and decisional Diffie-Hellman assumption) in such a way that if $\mathcal{A}$ manages to break protocol $\mathcal{P}$, then *at least* one of these algorithms succeeds in solving an underlying primitive. Since these primitives are widely deemed intractable, no probabilistic polynomial time (PPT) algorithm can succeed in breaking any one of them and hence protocol $\mathcal{P}$ remains secure (i.e., semantic security is preserved).

The tactic adopted by Tsai et al. to prove semantic security of the session key is similar to that of [35], [41]. They construct a series of attacking games G_n ($n = 0, 1, \dots, 5$), starting with the real attack G_0 and ending in a game G_5 where $\mathcal{A}$'s advantage is 0, and for which they bound the difference in $\mathcal{A}$'s advantage between any two consecutive games. This yields a bound on $\mathcal{A}$'s advantage in attacking the original protocol $\mathcal{P}$. Though Tsai et al. incrementally define a series of games, yet, *their security reductions are far from rigorous and they have placed a substitute by subterfuge!* More specifically, they carried out a security proof for their *three-factor* version of the scheme with a security model which is only suitable for *two-factor* authentication. Without an appropriate security model (which is the cornerstone of a security reduction) employed, the proof is destined to fail.

Tsai et al. divides their security model into five cases (see [29, Section 3]), yet no case is related to the biometric factor. As is well known, biometrics (e.g., fingerprint and iris) are prone to various sophisticated attacks and shall never be deemed as a secure “black box” that is free from threats [12], [47], [48]. This means their security model is unfit for analyzing three-factor (i.e., smart card, password and biometric) schemes, yet Tsai et al. just make an attempt to accomplish this task, while leaving the two-factor version of their scheme un-analyzed. The second one of their five cases is the smart-card-loss case. In this case, $\mathcal{A}$ is essentially equipped with the ability C-1 & C-2ii (see Table 1), i.e., $\mathcal{A}$ is assumed to be able to control the communication channel and has breached the user's card. Just under this case, we have shown their two-factor version cannot achieve truly two-factor security, which implies, at least, that their three-factor version cannot achieve truly three-factor security—Once the smart card factor and the biometric factor are compromised, the password factor can be offline guessed. This indicates neither the two-factor version nor three-factor version of their scheme are sound.

4 CRYPTANALYSIS OF LI'S SCHEME

In the above-analyzed scheme, the feature of user un-traceability is achieved by using a public key encryption which randomizes the user's real identity in session-variant pseudonym identities. In contrast, the scheme discussed in this

TABLE 6
Additional Notations Used in Li's Scheme

Symbol	Description
ID_A	identity of user A
PW_A	password of user A
d_S	secret key of remote server S
U_S	public key of remote server S , where $U_S = d_S \cdot G$
K_x	secret key where $K = PW_A \cdot r_A \cdot U_S = (K_x, K_y)$
$E_{K_x}(\cdot)/D_{K_x}(\cdot)$	symmetric encryption/decryption with K_x

section adopts a completely difficult strategy: each party updates the user's session-variant pseudonym identity after having authenticated its counterpart. Though this strategy can indeed support user un-traceability, as we shall show in the following, it is highly impractical, for it introduces a serious vulnerability that greatly downgrades the usability of the scheme. Moreover, this scheme also fails to provide truly two-factor security which is the most essential goal a two-factor shall achieve.

4.1 Review of Li's Scheme

In 2013, Li [33] proposed two ECC-based two-factor authentication schemes, one with user anonymity and the other without user anonymity. Here we are only interested in the one with user anonymity. This scheme consists of four phases: registration, authentication, password update and user eviction. For clarity, the intuitive abbreviations are listed in Table 4 and some additional ones in Table 6.

4.1.1 Registration Phase

The registration phase involves the following operations:

- 1) User A chooses her identity ID_A , password PW_A and $r_A \in_R \mathbb{Z}_n^*$, and computes $U_A = PW_A \cdot r_A \cdot G$;
- 2) $A \Rightarrow S : \{ID_A, U_A\}$.
- 3) On receiving the registration message, server S selects a pseudo-identity IND_A for A , and creates an entry $(IND_A, U_A, \text{status-bit})$ in its database, where *status-bit* indicates the status of A . More specifically, when A has logged-in to S , the *status-bit* is set to 1, otherwise it is set to 0.
- 4) $S \Rightarrow A$: A smart card containing parameters $\{IND_A, G, h(\cdot), E_{K_x}(\cdot)/D_{K_x}(\cdot)\}$.
- 5) Upon receiving the smart card, user A keys r_A into the card, which means the card henceforth stores $\{IND_A, G, h(\cdot), E_{K_x}(\cdot)/D_{K_x}(\cdot), r_A\}$.

4.1.2 Authentication Phase

When A logs in to S , the following steps are involved:

- 1) A inserts her smart card into the card reader, and inputs her password PW_A .
- 2) The smart card retrieves r_A , generates $r'_A \in_R \mathbb{Z}_n^*$, computes $R_A = r_A \cdot U_S = r_A \cdot d_S \cdot G$, $W_A = r_A \cdot r_A \cdot PW_A \cdot G$, $U'_A = PW_A \cdot r'_A \cdot G$ and $K = PW_A \cdot r_A \cdot U_S = (K_x, K_y)$.
- 3) $A \rightarrow S : \{IND_A, E_{K_x}(IND_A, R_A, W_A, U'_A)\}$.
- 4) Upon receiving the login request, S computes the decryption key K_x by computing $K = d_S \cdot U_A = PW_A \cdot$

$r_A \cdot d_S \cdot G = (K_x, K_y)$ and decrypts $E_{K_x}(IND_A, R_A, W_A, U'_A)$ to reveal $\{IND_A, R_A, W_A, U'_A\}$. Then S compares decrypted IND_A with received IND_A and $\hat{e}(d_S \cdot R_A, U_A)$ with $\hat{e}(W_A, U_S)$, respectively. If either is unequal, S rejects. Otherwise, the server will consider A as a legitimate user, which is justified by the following equalities:

$$\begin{aligned}\hat{e}(d_S \cdot R_A, U_A) &= \hat{e}(d_S \cdot r_A \cdot G, r_A \cdot pw_A \cdot G) \\ &= \hat{e}(G, G)^{r_A \cdot r_A \cdot pw_A \cdot d_S}\end{aligned}$$

$$\begin{aligned}\hat{e}(W_A, U_S) &= \hat{e}(r_A \cdot r_A \cdot pw_A \cdot G, d_S \cdot G) \\ &= \hat{e}(G, G)^{r_A \cdot r_A \cdot pw_A \cdot d_S}.\end{aligned}$$

- 5) S proceeds to generate a new pseudonym identity IND_A for A , selects $r_S \in_R \mathbb{Z}_n^*$, computes $W_S = r_S \cdot U_S = r_S \cdot d_S \cdot G$ and the session key $sk = r_S \cdot d_S \cdot W_A$.
- 6) $S \rightarrow A: \{W_A + W_S, h(W_S \| U'_A \| sk \| IND'_A), E_{sk}(IND'_A)\}$.
- 7) A derives W_S by subtracting W_A from $(W_A + W_S)$, computes $sk = r_A \cdot r_A \cdot PW_A \cdot W_S$, and gets IND'_A by decrypting $E_{sk}(IND'_A)$ using sk . A checks whether the hashed result of $\{W_S \| U'_A \| sk \| IND'_A\}$ equals the received $H(W_S \| U'_A \| sk \| IND'_A)$. If they are equal, A is assured that S is authentic and replaces $\{r_A, IND_A\}$ with $\{r'_A, IND'_A\}$.
- 8) $A \rightarrow S: \{IND_A, h(sk \| IND'_A)\}$.
- 9) S checks whether the hashed result of $\{sk \| IND'_A\}$ equals the received $h(sk \| IND'_A)$. If it holds, S granted A 's login request and replaces $\{IND_A, U_A\}$ with $\{IND'_A, U'_A\}$.

4.1.3 Password Change Phase

The password change phase is provided to allow users to change their passwords freely (but not locally). When A wants to change her password, she first needs to go through the above authentication phase to make sure that the input password is valid and then can change it to a new one.

4.1.4 User Eviction Phase

In case the period of validity of user A expires, then the user can be evicted by the remote server S by deleting (IND_A, U_A) from its backend database. Thereafter, A can no longer use IND_A and U_A to login S .

4.2 Cryptanalysis of Li's Scheme

In [33], Li demonstrated that Islam-Biswas's scheme is vulnerable to several serious attacks such as offline password guessing and stolen-verifier, and to overcome the identified weaknesses, a new scheme with user anonymity was further presented. Besides its high efficiency due to the use of ECC, this scheme is claimed (and heuristically argued) to provide robust security (i.e., provision of SR1~SR7) and support five attractive properties (i.e., DA1~DA5). Accordingly, it seems very appealing and shows great application potential. However, after a careful investigation, we find it still far from practical—it is of poor usability and fails to achieve two-factor

security under their non-tamper resistance assumption of the smart cards.

4.2.1 Smart Card Loss Attack

Evidently, the most essential goal of a two-factor authentication scheme is to provide two-factor security, which means a compromise of either the password factor or the smart card factor will not lead to the compromise of the system. As pointed out in [30], [32], to date few schemes have achieved this "precious" goal. Once more, Li's attempt [33] ends in vain, as we will show how an attacker in possession of a user's smart card can recover the password with the help of an automated procedure.

Suppose an adversary $\mathcal{A}$ has somehow obtained (stolen or picked up) user A 's smart card and extracted the secret information $\{IND_A, r_A, U_S\}$ by using side-channel attacks [19], [20], [21] herself (or with recourse to professional labs), and then $\mathcal{A}$ returns the breached card back to A without A 's awareness. Note that these assumptions are quite practical as discussed in Sections 2.1 and 3.2.1. Once user A uses the breached smart card to login, the attacker can intercept A 's login request $\{IND_A, E_{K_x}(IND_A, R_A, W_A, U'_A)\}$ and then obtains PW_A as follows:

- Step 1. Guesses the value of PW_A to be PW_A^* from the dictionary space $\mathcal{D}_{pw}$.
- Step 2. Computes $K^* = PW_A^* \cdot r_A \cdot U_S = (K_x^*, K_y^*)$, where r_A, U_S are extracted from A 's smart card.
- Step 3. Derives IND_A^* by decrypting the previously intercepted $E_{K_x}(IND_A, R_A, W_A, U'_A)$ using K_x^* .
- Step 4. Verifies the correctness of PW_A^* by checking if IND_A^* is equal to the extracted IND_A .
- Step 5. Repeats Step 1 ~ 4 of this procedure until the correct value of PW_A is found.

Let $|\mathcal{D}_{pw}|$ denote the size of password space $\mathcal{D}_{pw}$. The time complexity of the above attacking procedure is $\mathcal{O}(|\mathcal{D}_{pw}| * (2T_P + T_S))$, where T_P is the running time for ECC point multiplication and T_S the running time for symmetric decryption. In other words, the time for $\mathcal{A}$ to recover U_i 's password is linear with $|\mathcal{D}_{pw}|$, and thus our attack is quite effective. Furthermore, in practice users generally choose common and relatively weak passwords, and thus $\mathcal{D}_{pw}$ is very restricted, e.g., $|\mathcal{D}_{pw}| \leq 10^6$ [43]. According to the operation timings reported in Table 5, $\mathcal{A}$ can accomplish the above procedure in seconds on a common PC.

Our attack implies that once the smart-card factor is compromised, the remaining password factor can be offline guessed by an automated attacking procedure, which is the so-called offline password guessing attack [32]. Since then, there is no way to prevent $\mathcal{A}$ from impersonating A to enjoy the system's services/resources, unless A re-registers with the server. This suggests that Li's scheme is essentially not a truly two-factor scheme and provides no better security than the original scheme (i.e., Islam-Biswas's scheme).

4.2.2 De-Synchronization Attack

To provide user un-traceability, a number of dynamic-ID based two-factor protocols (e.g., [24], [29], [35]) construct a new pseudo-identity for the user in each session by using cryptographic methods (e.g., public encryption algorithm)

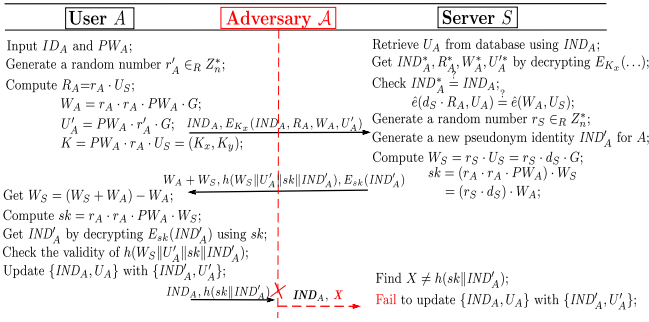


Fig. 2. De-synchronization attack on Li's scheme.

during the login process, while the other dynamic-ID based authentication protocols (e.g., [11], [22], [49]) adopt a quite different strategy: a new user pseudo-identity for the next login request is constructed during the current authentication process. In the latter strategy, it is evident that the new user pseudo-identity (which will be used for the next login request) shall be stored somewhere on the user side, and to recognize this user in the following protocol run, the server also needs to maintain a copy of the user's new pseudo-identity after the current protocol run. So the synchronization of this new pseudo-identity between the user side and the server side is crucial for their following successful protocol runs. However, there is no easy way to make sure that this synchronization is well maintained. As we will show and discuss in the following, a determined attacker can always somehow break this synchronization and render the user unable to login *ever since*, which suggests the infeasibility of the latter strategy (i.e., by employing a synchronization mechanism).

Let us see a concrete example. Suppose user A has performed Step 7 of the authentication phase (see Section 4.1.2) and sends $\{IND_A, h(sk \| IND'_A)\}$ to S as specified, which means A has replaced $\{r_A, IND_A\}$ with $\{r'_A, IND'_A\}$ in her card memory. Before $\{IND_A, h(sk \| IND'_A)\}$ reaches S , A intercepts this message and alters it to $\{IND_A, X\}$, where X is a randomly selected value. In Step 8 of the authentication phase, S will find $X \neq h(sk \| IND'_A)$, and surely enough, will reject A 's login request and refuse to update $\{U_A, IND_A\}$ to $\{U'_A, IND'_A\}$ in its backend database. Consequently, the consistency of the user pseudo-identity between A and S is broken. Thereafter, A will send IND'_A to S in her login requests, yet S stores the old IND_A and will always reject A due to the mismatch $IND'_A \neq IND_A$.

The above attack, as summarized in Fig. 2, is practically effective, for the attacker is only required to alter a single protocol transcript (i.e., the third message from A to S) and then can completely destroy the "synchronization" between the user and the server. In other words, there are no other expensive operations involved such as reverse engineering and power analysis. Moreover, it is not difficult to see that, instead of altering this protocol transcript, A might equally attain her end by simply blocking this protocol transcript.

It also should be noted that, although this vulnerability appears rather simple, it cannot be well addressed just with minor revisions. One may think that, if an additional "ack" message is sent back to user A and only when A has received this "ack" should she update $\{r_A, IND_A\}$ to

$\{r'_A, IND'_A\}$, then the above presented attack will not work. Admittedly, this is true, but what will happen if A now simply alter (or block) this "ack" message? Apparently, in this case, user A will wait for an "ack" message which never comes, failing to replace $\{r_A, IND_A\}$ with $\{r'_A, IND'_A\}$. On the other hand, the server S has already updated $\{U_A, IND_A\}$ to $\{U'_A, IND'_A\}$ before sending this "ack". Similarly, any attempt to overcome this vulnerability by adding new protocol flow(s) will be doomed to failure.

Another possible (defensive) solution one may think is to store both the old and new pseudo-identities on the smart card and/or the server side, and the old pseudo-identity is put to use whenever the new one fails to work (i.e., a de-synchronization has occurred). Regrettably, such a solution may bring other issues. One prominent problem is that user un-traceability will be violated once the adversary blindly blocks a new conversation initiated by the same user.

Remark 2. We have analyzed more than 120 recently proposed two-factor schemes and more than 50 anonymous two-factor schemes (some of our recent cryptanalysis results include [2], [32], [35], [39]), and observed that all these schemes that employ a similar synchronization mechanism to that of Li's scheme [33] to maintain the consistency of user pseudo-identities are subject to de-synchronization problem without no exception. Some other problematic ones include [11], [22], [49]. The above attack reveals the unsoundness of these schemes. Unfortunately, as stated in [50], [51], the widely used formal methods (e.g., random oracle model, BAN logic) can not capture such structural mistakes, and assuring soundness of authentication protocols still remains an open issue.

5 EXPLORATION OF THE RELATIONSHIPS AMONG EVALUATION CRITERIA

In this section, we first summarize and investigate the available ways to achieve user anonymity. Then, we explicate the definitions of some criteria in Madhusudhan-Mittal's set [25] to provide a clear basis for investigating their inner relationships. Further, using these two schemes examined earlier in this paper as case studies and building on the previous cryptanalysis results [2], [30], [31], [32], [38], [39], we for the first time show that it is unlikely to construct an "ideal" dynamic ID-based two-factor authentication scheme that satisfies all the criteria in Madhusudhan-Mittal's set [25], and provide a negative answer to the question raised by Huang et al. [30]. Lastly, considering the promising applications of formal methods, we discuss what the role of "provable security" is in breaking the vicious circle of "attack-fix-attack-fix".

5.1 Ways to Achieve User Anonymity

Intuitively, there are two broad approaches to implement the "dynamic ID technique": (1) Making use of cryptographic primitives (e.g., symmetric-key operations like that of [16], [17], CCA-2 public key encryption [52] like that of [29], [35], ring/group signatures [53], [54] or attribute-based signatures [55]); and (2) Exploiting some non-cryptographic mechanisms (e.g., pre-loading a pseudo-IDs pool like that

of [56], or synchronization mechanism like that of [11], [33]). However, it is not difficult to see that, the idea of pre-loading a large pseudo-IDs pool on a resource-limited smart card is practically infeasible, while most anonymous two-factor schemes that employ some types of synchronization mechanism, which has been discussed in Section 4, are prone to a fatal usability problem.

As non-cryptographic mechanisms do not seem to work, the cryptographic approach remains the only alternative to preserve user anonymity in two-factor authentication. Since ring/group signatures often need the support of public key infrastructure (PKI) and their computation overhead grows linearly with the ring/group size, neither of them could readily be used in two-factor authentication schemes. Attribute-based signatures are free from the burden of PKI, yet there are generally a number of expensive bilinear pairing operations involved, which may be unsuitable for implementation on low-power smart cards. What's more, as pointed out in [13], how to keep secret the signing key on the user side is a non-trivial issue when smart cards are assumed that they can be tampered when lost.

To the best of knowledge, all this for the first time well explains why most anonymous two-factor schemes [15], [16], [17], [18], [23], [29], [31], [33], [35] prefer to only employ some symmetric cryptographic primitives (e.g., hash functions, XOR operations, symmetric encryption) or a few comparatively lightweight public-key operations (e.g., modular exponentiations and elliptic curve point multiplications). Accordingly, we further classify these anonymous schemes into two categories: (1) Symmetric-key-based ones; and (2) Public-key based ones. We say the former kind of schemes is with the property “DA5-SymmetricKey” and the latter kind of schemes provides the property “DA5-PublicKey”. In a recent work [39], we manage to show that these two properties do have intrinsic (but subtle) relationships with the security requirement SR6 (i.e., Resistance to smart card loss attack), which will be elaborated later.

5.2 Further Explications of Some Criteria

A scheme supporting property DA1 requires that there is no password-related verification data stored on the server, ensuring that a compromise of the server will not lead to the disclosure of all the users' passwords. Since the first scheme with DA1 was proposed by Yang and Shieh [4] in 1999, DA1 has become one of the most basic design goals of two-factor schemes [26], [27]. Like the scheme investigated in Section 3, a number of schemes (e.g., [15], [16], [18], [28]) attempting to achieve DA1 advocate that the server shall only keep some secret key(s) for verifying the users and there be no other user-specific data stored on the server. On the contrary, the other schemes (e.g., the scheme examined in Section 4 and [17], [22], [35]) with DA1 store some non-security-critical user-specific information such as $\{ID_i, T_{reg}\}$ on the server side, where ID_i is user's identity and T_{reg} user's registration time. We say the former kind of schemes provides the property “DA1-Strong” and the latter kind of schemes supports the property “DA1-Weak”.

In password-based authentication, besides free user password choice (DA2), it is a universally accepted practice that users shall regularly change their passwords.

Accordingly, as stated in Section 2.1, the password change phase has been a basic phase in any two-factor scheme. Obviously, this phase may either enable a user to locally change her password or require a user to interact with the server in order to change her password. As investigated in Section 3.2.2, a scheme that facilitates the user to locally change her password but does not support secure password change is prone to smart card loss attack (i.e., no provision of SR6), while a scheme that facilitates the user to locally change her password as well as supporting secure password change is prone to the same threat. For ease of presentation, we say the former scheme is with attribute “DA2-Local-Secure”, the latter one with the attribute “DA2-Local-Insecure”. In addition, for a scheme that does not support local password change, we say it provides the attribute “DA2-Interactive”.

5.3 Relationships among the Evaluation Criteria

In this work, we mainly focus on the following three most basic relationships among the evaluation criteria: symbiotic, mutually exclusive and implicative, denoted by ∞ , $\otimes$ and $\triangleright$, respectively. More specifically, DA_i and SR_j being of a symbiotic relation (i.e., $DA_i \infty SR_j$) means if either one is held by the scheme, both will be held; DA_i and SR_j are mutually exclusive (i.e., $DA_i \otimes SR_j$) if and only if at any time, at most one of them is held by the scheme; DA_i and SR_j being of an implicative relation means that either (1) whenever DA_i is held by the scheme, SR_j is held by the scheme (i.e., $DA_i \triangleright SR_j$) or (2) whenever SR_j is held by the scheme, DA_i is held by the scheme (i.e., $SR_j \triangleright DA_i$). Our observations are summarized in Table 7 and detailed as follows:

1) **DA1-Strong $\otimes$ DA9.** A scheme that supports the property DA1-Strong means there is no user-specific (or card-specific) information stored on the server. However, even if the authorized time of a card has expired, how can the server (e.g., the schemes in [16], [28], [29]) tell apart a valid card from an expired card? As far as we know, there is no way for the server S to accomplish this aim. One may think that, for those expired cards, S adds a piece of valid time information as well as a digital signature for this data on the card, then by verifying the signature, S can tell apart valid cards from expired ones; For those revoked cards, S can keep a revocation list, as it does in a traditional certificate authority. However, in this way, it requires S to store a revocation list, which defeats the purpose of storing no user-specific data on S (i.e., DA1-Strong).

2) **DA1-Strong $\triangleright$ SR7.** Since there is actually no verifier stored in the server, of course, stolen-verifier-attack can be prevented. On the contrary, a scheme free from stolen-verifier-attack may support DA1-Weak but not DA1-Strong.

3) **DA1-Weak $\triangleright$ SR7.** Since there is actually no security-critical verifier stored in the server, stolen-verifier-attack can be eliminated. On the contrary, a scheme free from stolen-verifier-attack may support DA1-Strong but not DA1-Weak.

4) **DA2-Local-Secure ∞ DA10.** A scheme that supports DA2-Local-Secure means a user can locally and securely update her password, this implies that there are some password-related verifiers (e.g., $A_i = h(ID_i || PW_i)^{PW_i} \bmod p$ in [28], or $\{r, N = h(r || x) \times h(PW_i), Y = h(ID_i || h(r || x))\}$ in

TABLE 7
Relationships among the Criteria in Madhusudhan-Mittal's Set

Design Goals	DA1: No password verifier table	DA2: Password friendliness	DA3: No password reveal	DA4: Password dependent	DA5: Mutual authentication	DA6: Session key agreement	DA7: Forward secrecy	DA8: User anonymity	DA9: Smart card revocation	DA10: timely typo detection	SR1: Resist to DoS attack	SR2: Resist impersonation attack	SR3: Resist parallel session attack	SR4: Resist password guessing attack	SR5: Resist replay attack	SR6: Resist smart card loss attack	SR7: Resist stolen-verifier attack	SR8: Resist reflection attack	SR9: Resist insider attack
DA1-Strong	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA1-Weak	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA2-Local-Secure	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA2-Local-Insecure	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA2-Interactive	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA3	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA4	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA5-SymmetricKey	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA5-PublicKey	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA6	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA7	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA8	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA9	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
DA10	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

Note: $X \ast Y$ means the relation between X and Y is unknown (probably independent); $X \infty Y$ means a symbiotic relation between X and Y ; $X \triangleright Y$ means Y is implied by X ; $X \otimes Y$ means a mutually exclusive relation between X and Y .

[34]) stored in the card memory, and these password-related verifiers can just be used to check whether the user has accidentally input a wrong password when login.

5) DA2-Local-Secure $\otimes$ SR6. A scheme that supports DA2-Local-Secure means a user can locally and securely update her password, this implies that there are some password-related verifiers (e.g., $A_i = h(ID_i \| PW_i)^{PW_i} \bmod p$ in [28], or $\{r, N = h(r \| x) \times h(PW_i), Y = h(ID_i \| h(r \| x))\}$ in [34]) stored in the card memory, and these password-related verifiers can just be used to check whether a guessed password is right or not once an attacker has obtained the card and extracted these password-related verifiers.

6) DA2-Local-Insecure $\otimes$ DA10. This relationship can be obtained directly from the relationship that DA2-Local-Secure ∞ DA10.

7) DA2-Local-Insecure $\otimes$ SR6. A two-factor scheme that supports DA2-Local-Insecure means a user can locally change her password while there is no password-related verifier stored in the card memory. Apparently, this scheme can not prevent an attacker from easily changing the password, as shown in Section 3.2.2. On the other hand, a scheme supports SR6 means an attacker shall not easily change the password when obtaining the card, indicating DA2-local-insecure is not supported in the scheme.

8) DA2-Interactive $\otimes$ DA10. A scheme attains the feature DA2-Interactive (e.g., the schemes in [11], [33]) means that the user is required to change her password by interacting with the server. This implies that there is no password-related verifier stored on the card. Without such data, there is no information that the card can use to check whether or not the user has accidentally input a wrong password when login. This indicates that a scheme supporting DA2-Interactive cannot provide DA10, and vice versa.

9) DA5 $\triangleright$ SR2, SR3, SR5 and SR8. A scheme supporting mutual authentication (i.e., DA5) means an attacker can not impersonate either the user or the server. This excludes the

possibility of impersonation attack, parallel session attack, replay attack and reflection attack. Otherwise, DA5 can not be assured. On the other hand, neither the achievement of SR2 (nor, SR3, SR5 SR8) indicates the achievement of DA5. For example, a two-factor scheme (e.g., [41]) achieves SR8 may still be prone to impersonation attack, invalidating the criteria DA5 and SR2.

10) DA5-SymmetricKey $\otimes$ DA7, DA8 and SR6. A scheme that supports DA5-SymmetricKey means mutual authentication is achieved by only using symmetric-key techniques. According to [2], [39], under the assumption C-2 as listed in Table 1, two-factor authentication schemes that do not employ public-key primitives are intrinsically unable to provide DA7, DA8 and SR6.

11) DA10 $\otimes$ SR6. A scheme that supports DA10 (e.g., [28], [31]) means the smart card can timely detect whether the user has accidentally input a wrong password when login. To this end, there should be some password-related verifier(s) stored on the card. In this case, an attacker can perform smart card lost attack just by exploiting this data.

12) SR6 $\triangleright$ DA4. According to the definitions given in [25], DA4 is entirely incorporated into SR6: both SR6 and DA4 concern the case in which a user has lost her card, yet SR6 requires that $\mathcal{A}$ shall not be able to perform impersonation attack, offline guessing attack or easily change the password (see Section 3.2.2), while what a scheme supporting DA4 can only guarantee is that $\mathcal{A}$ needs to know the password to impersonate the user (but $\mathcal{A}$ may perform other malicious operations such as easily changing the password). For example, the scheme in [16] is a typical one that fails to achieve both DA4 and SR6, while the schemes in [17], [18], [23] achieve DA4 but fail to provide SR6.

From Table 7, one can see that there is always a mutually exclusive relationship (denoted by $\otimes$) among "DA2-*" and some other criteria. More specifically, both DA2-Local-Secure and DA2-Local-Insecure are mutually exclusive

with SR6, while DA2-Interactive is mutually exclusive with DA10. This means no matter how the user changes her password (i.e., locally or interactively), either SR6 or DA10 definitely can not be achieved, which indicates it is unlikely to construct an “ideal” scheme that satisfies all the criteria in Madhusudhan-Mittal’s evaluation set [25]. In particular, the relationships among DA2-Local-Secure, DA2-Local-Insecure and SR6 suggest a negative answer to the question left by Huang et al. [30]—“whether or not there exist secure smart-card-based password authentication protocols that the password-changing phase does not need any interaction with the server”?

5.4 The Role of “Provable Security”

For several decades, protocol designers worked by trials and errors. A protocol was proposed, and then the community tried to break and fix it. It turned out that many protocols were compromised a number of years after they were first suggested. This unsatisfactory situation was not ameliorated until the early 1980s by the seminal work of Goldwasser and Micali [57], followed by a series of influential works [5], [36]. These works suggest that security could be “proved” under well known complexity-theoretic assumptions (e.g., the intractability of CDH problem), and the methodology they identified is thereafter called “provable security”. This methodology has been proved especially useful in eliminating redundancies in the list of attacks on a protocol. For example, most of the attacking-oriented goals listed in Table 2 may be reduced to only two formal ones, namely, semantic security and mutual authentication [35]. Furthermore, security goals defined in a formal model are more precise in capturing the security provisions offered by a protocol than that of a heuristic model. Consequently, provable security has become an indispensable tool in analyzing and evaluating new cryptographic protocols.

However, provable security has its limitations. Generally, the process of providing “provable security” for a protocol entails five stages: (1) Definition of adversarial model; (2) Statement of security goals; (3) Specification of cryptographic assumptions; (4) Description of protocol; and (5) Reductionist proof. It follows that any provably secure protocol meets its goals within some security model under some cryptographic assumption(s), rather than the mere claim that such-and-such a protocol achieves provable security. Past research over the last thirty decades has told us that, a security proof is highly prone to be fallacious due to the adoption of an insufficient security model which fails to capture all the realistic capabilities of the adversary or due to a flawed/non-tight security reduction, and “the field of provable security is as much an art as a science” [58], [59]. Our past experience of cryptanalysis of two-factor schemes over the last two decades has revealed that, most of the two-factor schemes (e.g., the ones in [22], [24], [34], [41]) that are equipped with a formal proof have been found severely problematic shortly after they were presented. Our “smart card loss attack I” (see Section 3.2.1) on Tsai et al.’s protocol perfectly demonstrates that, having a formal (but insufficient) security model and designing a “proven secure” protocol in that model are no panacea for assuring actual security. While formal methods are

often misused and reductionist security proofs are usually very intricate, turgid and prone to errors, particular care shall be given when conducting security proofs for complex cryptographic cryptosystems like two-factor schemes.

It is also worth noting that, many attacking scenarios are difficult to be captured in a formal adversarial model. For example, the “smart card loss attack II” (see Section 3.2.2) on Tsai et al.’s protocol and the “de-synchronization attack” (see Section 4.2.2) on Li’s protocol, as far as we know, cannot be captured in any existing model. While these practical attacks cannot be modelled in current models, it is crucial that protocol designers are fully aware of such damaging threats. What’s more, as shown in [51], [60], even if the security model employed is the right one at the present, the correctness of a security proof largely depends on the prover’s attacking experience. Last but not the least, even if the security model is accurate and the security proof is correct, the features (functionalities) of a protocol can hardly be analyzed or assured by the methodology of provable security. All this highlights the critical role that old-fashioned cryptanalysis continues to play in establishing confidence in the security and versatility of a cryptographic protocol, suggesting the importance and necessity of this work.

6 CONCLUSION

In this paper, we have investigated the question of whether it is possible to build an “ideal” anonymous two-factor authentication scheme that satisfies all the criteria listed in Madhusudhan-Mittal’s evaluation set? By cryptanalyzing two foremost anonymous two-factor schemes as case studies, we uncover several subtleties and challenges in designing this type of schemes, and explore the relationships among the criteria. Our results highly indicate a negative answer to the examined question. Most essentially, we find that, a scheme supporting local user password change is unlikely to achieve “SR6: resistance to smart card loss attack”, while a scheme not supporting local user password change is unlikely to provide the property of “DA10: timely typo detection”. This presents an unavoidable usability-security tradeoff, thereby also suggesting a negative answer to the open question raised by Huang et al. [30].

We believe this work provides a better understanding of the underlying evaluation metric for anonymous two-factor schemes, which is of fundamental importance for security engineers to make their choices correctly and for protocol designers to develop practical schemes with better usability-security tradeoffs. We leave for future work the question of evaluating practical effectiveness of the proposed “fuzzy-verifiers” by using recently disclosed large-scale real-life password data-sets like the 50 million “Evernote” dataset and the 6.4 million “LinkedIn” dataset.

ACKNOWLEDGMENTS

The authors are grateful to the anonymous reviewers for their valuable comments that highly improve the readability and completeness of the paper. This research was supported by the National Natural Science Foundation of China (NSFC) under Grants Nos. 61472016 and 61370224.

REFERENCES

- [1] M. Bond, O. Choudary, and S. Murdoch, "Chip and skim: Cloning EMV cards with the pre-play attack," in *Proc. IEEE S&P 2014*, 2014, pp. 1–15.
- [2] D. Wang and P. Wang, "On the anonymity of two-factor authentication schemes for wireless sensor networks: Attacks, principle and solutions," *Comput. Netw.*, vol. 73, pp. 41–57, 2014.
- [3] N. Gunson, D. Marshall, H. Morton, and M. Jack, "User perceptions of security and usability of single-factor and two-factor authentication in automated telephone banking," *Comput. Security*, vol. 30, no. 4, pp. 208–220, 2011.
- [4] W.-H. Yang and S.-P. Shieh, "Password authentication schemes with smart cards," *Comput. Security*, vol. 18, no. 8, pp. 727–733, 1999.
- [5] M. Bellare, D. Pointcheval, and P. Rogaway, "Authenticated key exchange secure against dictionary attacks," in *Proc. 19th Int. Conf. Theory Appl. Cryptographic Tech.*, 2000, vol. 1807, pp. 139–155.
- [6] J. Katz, R. Ostrovsky, and M. Yung, "Efficient and secure authenticated key exchange using weak passwords," *J. ACM*, vol. 57, no. 1, pp. 1–41, 2009.
- [7] 50 million compromised in evernote hack. (2013, Mar.) [Online]. Available: <http://www.cnn.com/2013/03/04/tech/web/evernote-hacked/>
- [8] P. Sean, *LinkedIn Passwords Leaked Online: Hackers are beginning to decrypt 6.4 million passwords*. (2012, Jun. 6) [Online]. Available: <http://www.webpronews.com/linkedin-passwords-leaked-online-2012-06>
- [9] Heartbleed—openssl zero-day bug leaves millions of websites vulnerable. (2014, Apr.) [Online]. Available: <http://www.tuicool.com/articles/yyUfUz>
- [10] G. Yang, D. Wong, H. Wang, and X. Deng, "Two-factor mutual authentication based on smart cards and passwords," *J. Comput. Syst. Sci.*, vol. 74, no. 7, pp. 1160–1172, 2008.
- [11] X. Li, W. Qiu, D. Zheng, K. F. Chen, and J. Li, "Anonymity enhancement on robust and efficient password-authenticated key agreement using smart cards," *IEEE Trans. Ind. Electron.*, vol. 57, no. 2, pp. 793–800, Feb. 2010.
- [12] X. Huang, Y. Xiang, A. Chonka, J. Zhou, and R. H. Deng, "A generic framework for three-factor authentication: Preserving security and privacy in distributed systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 8, pp. 1390–1397, Aug. 2011.
- [13] Y. G. Wang, "Password protected smart card and memory stick authentication against off-line dictionary attacks," in *Proc. Inform. Security Privacy Conf.*, 2012, vol. 376, pp. 489–500.
- [14] F. Zhu, S. Carpenter, and A. Kulkarni, "Understanding identity exposure in pervasive computing environments," *Pervasive Mob. Comput.*, vol. 8, no. 5, pp. 777–794, 2012.
- [15] M. Das, A. Saxena, and V. Gulati, "A dynamic ID-based remote user authentication scheme," *IEEE Trans. Consum. Electron.*, vol. 50, no. 2, pp. 629–631, May 2004.
- [16] Y. Wang, J. Liu, F. Xiao, and J. Dan, "A more efficient and secure dynamic ID-based remote user authentication scheme," *Comput. Commun.*, vol. 32, no. 4, pp. 583–585, 2009.
- [17] M. Khan and S. Kim, "Cryptanalysis and security enhancement of a more efficient & secure dynamic ID-based remote user authentication scheme," *Comput. Commun.*, vol. 34, no. 3, pp. 305–309, 2011.
- [18] Y.-F. Chang, W.-L. Tai, and H.-C. Chang, "Untraceable dynamic-identity-based remote user authentication scheme with verifiable password update," *Int. J. Commun. Syst.*, 2013, Doi: 10.1002/dac.2368.
- [19] T. H. Kim, C. Kim, and I. Park, "Side channel analysis attacks using am demodulation on commercial smart cards with seed," *J. Syst. Softw.*, vol. 85, no. 12, pp. 2899–2908, 2012.
- [20] K. Nohl, D. Evans, S. Starbug, and H. Plötz, "Reverse-engineering a cryptographic rfid tag," in *Proc. 17th Conf. Security Symp.*, 2008, pp. 185–193.
- [21] A. Barenghi, L. Breveglieri, and D. Naccache, "Fault injection attacks on cryptographic devices: Theory, practice, and countermeasures," *Proc. IEEE*, vol. 100, no. 11, pp. 3056–3076, Nov. 2012.
- [22] R. C. Wang, W. S. Juang, and C. L. Lei, "Robust authentication and key agreement scheme preserving the privacy of secret key," *Comput. Commun.*, vol. 34, no. 3, pp. 274–280, 2011.
- [23] K. Xue, P. Hong, and C. Ma, "A lightweight dynamic pseudonym identity based authentication and key agreement protocol without verification tables for multi-server architecture," *J. Comput. Syst. Sci.*, vol. 80, no. 1, pp. 195–206, 2014.
- [24] S. H. Wu, Y. F. Zhu, and Q. Pu, "Robust smart-cards-based user authentication scheme with user anonymity," *Security Commun. Netw.*, vol. 5, no. 2, pp. 236–248, 2012.
- [25] R. Madhusudhan and R. Mittal, "Dynamic ID-based remote user password authentication schemes using smart cards: A review," *J. Netw. Comput. Appl.*, vol. 35, no. 4, pp. 1235–1248, 2012.
- [26] I. Liao, C. Lee, and M. Hwang, "A password authentication scheme over insecure networks," *J. Comput. Syst. Sci.*, vol. 72, no. 4, pp. 727–740, 2006.
- [27] C. Tsai, C. Lee, and M. Hwang, "Password authentication schemes: current status and key issues," *Int. J. Netw. Security*, vol. 3, no. 2, pp. 101–115, 2006.
- [28] X. Li, J. Niu, M. Khurram Khan, and J. Liao, "An enhanced smart card based remote user password authentication scheme," *J. Netw. Comput. Appl.*, vol. 36, no. 5, pp. 1365–1371, 2013.
- [29] J.-L. Tsai, N.-W. Lo, and T.-C. Wu, "Novel anonymous authentication scheme using smart cards," *IEEE Trans. Ind. Inform.*, vol. 9, no. 4, pp. 2004–2013, Nov. 2013.
- [30] X. Huang, X. Chen, J. Li, Y. Xiang, and L. Xu, "Further observations on smart-card-based password-authenticated key agreement in distributed systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 7, pp. 1767–1775, Jul. 2014.
- [31] S. Kumari and M. K. Khan, "More secure smart card-based remote user password authentication scheme with user anonymity," *Security Commun. Netw.*, 2014.
- [32] D. Wang, and P. Wang, "Offline dictionary attack on password authentication schemes using smart cards," in *Proc. 16th Inform. Security Conf.*, 2013, pp. 1–16.
- [33] C.-T. Li, "A new password authentication and user anonymity scheme based on elliptic curve cryptography and smart card," *IET Inform. Security*, vol. 7, no. 1, pp. 3–10, 2013.
- [34] K. H. Yeh, C. Su, N. W. Lo, Y. Li, and Y. X. Hung, "Two robust remote user authentication protocols using smart cards," *J. Syst. Softw.*, vol. 83, no. 12, pp. 2556–2565, 2010.
- [35] D. Wang, P. Wang, C. Ma, and Z. Chen, "iPass: Robust smart card based password authentication scheme against smart card loss problem," *J. Comput. Syst. Sci.*, In press, 2014, full version [Online]. Available: <http://eprint.iacr.org/2012/439.pdf>.
- [36] R. Canetti and H. Krawczyk, "Analysis of key-exchange protocols and their use for building secure channels," in *Proc. Int. Conf. Theory Appl. Cryptographic Tech.: Adv. Cryptol.*, 2001, vol. 2045, pp. 453–474.
- [37] J. Bonneau, M. Just, and G. Matthews, "What's in a name?" in *Proc. FC*, 2010, vol. 6052, pp. 98–113.
- [38] M. Scott, Cryptanalysis of a recent two factor authentication scheme. (2012). Cryptology ePrint Archive, Rep. 2012/527 [Online]. Available: <http://eprint.iacr.org/2012/527.pdf>
- [39] C. Ma, D. Wang, and S. Zhao, "Security flaws in two improved remote user authentication schemes using smart cards," *Int. J. Commun. Syst.*, 2012, Doi: 10.1002/dac.2468.
- [40] E. Morse, M. Theofanos, Y. Choong, C. Paul, and A. Zhang, "Usability of piv smartcards for logical access," US Dept. Commerce, NIST, Gaithersburg, MD, USA, Tech. Rep. NIST-IR-7867, 2012.
- [41] J. Xu, W. Zhu, and D. Feng, "An improved smart card based password authentication scheme with provable security," *Comput. Stand. Inter.*, vol. 31, no. 4, pp. 723–728, 2009.
- [42] *Miracl library*, Shamus Software Ltd. (2013, May) [Online]. Available: <http://www.shamus.ie/index.php?page=home>
- [43] J. Bonneau, "The science of guessing: Analyzing an anonymized corpus of 70 million passwords," in *Proc. IEEE Symp. Security Privacy*, 2012, pp. 538–552.
- [44] *Standards for Efficient Cryptography, SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0*, Certicom Research. (2010, Jan.) [Online]. Available: <http://www.secg.org/download/aid-784/sec2-v2.pdf>
- [45] J. Nam, S. Kim, and D. Won, "Security analysis of a nonce-based user authentication scheme using smart cards," *IEICE Trans. Fund. Electron. Comm. Comput. Sci.*, vol. 90, no. 1, pp. 299–302, 2007.
- [46] M. Alsaleh, M. Mannan, and P. Van Oorschot, "Revisiting defenses against large-scale online password guessing attacks," *IEEE Trans. Depend. Secur. Comput.*, vol. 9, no. 1, pp. 128–141, Jan./Feb. 2012.
- [47] *Thinking Putty defeats Fingerprint Scanners! Dan's Data* (2013, Sep.) [Online]. Available: <http://www.puttyworld.com/thinputdeffi.html>

- [48] X. Huang, Y. Xiang, E. Bertino, J. Zhou, and L. Xu, "Robust multi-factor authentication for fragile communications," *IEEE Trans. Depend. Secur. Comput.*, 2013, Doi: 10.1109/TDSC.2013.2297110.
- [49] S. Kumari and M. K. Khan, "Cryptanalysis and improvement of a robust smart-card-based remote user password authentication scheme," *Int. J. Commun. Syst.*, 2013, Doi: 10.1002/dac.2590.
- [50] G.-L. Wang, J.-S. Yu, and Q. Xie, "Security analysis of a single sign-on mechanism for distributed computer networks," *IEEE Trans. Ind. Inform.*, vol. 9, no. 1, pp. 294–302, Feb. 2013.
- [51] F. Bao, "Security can only be measured by attacks. (2008) [Online]. Available: <http://wenku.baidu.com/view/ff14a31ea300a6c30c229f7a.html>
- [52] E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," in *Proc. CRYPTO 19th Annu. Int. Cryptol. Conf.*, 1999, vol. 1666, pp. 537–554.
- [53] D. Chaum and E. Heyst, "Group signatures," in *Proc. Workshop Theory Appl. Cryptographic Tech.*, 1991, vol. 547, pp. 257–265.
- [54] J. Ren, and L. Harn, "An efficient threshold anonymous authentication scheme for privacy-preserving communications," *IEEE Trans. Wireless Commun.*, vol. 12, no. 3, pp. 1018–1025, Mar. 2013.
- [55] H. Maji, M. Prabhakaran, and M. Rosulek, "Attribute-based signatures," in *Proc. CT-RSA*, 2011, pp. 376–392.
- [56] D. He, C. Chen, S. Chan, and J. Bu, "Secure and efficient handover authentication based on bilinear pairing functions," *IEEE Trans. Wireless Commun.*, vol. 11, no. 1, pp. 48–53, Jan. 2012.
- [57] S. Goldwasser and S. Micali, "Probabilistic encryption," *J. Comput. Syst. Sci.*, vol. 28, no. 2, pp. 270–299, 1984.
- [58] N. Kobitz and A. J. Menezes, "Another look at 'provable security'," *J. Cryptol.*, vol. 20, no. 1, pp. 3–37, 2007.
- [59] T.-Y. Li and G.-L. Wang, "Analyzing a family of key protection schemes against modification attacks," *IEEE Trans. Depend. Secur. Comput.*, vol. 8, no. 5, pp. 770–776, Sep./Oct. 2011.
- [60] A. Menezes. (2012). Another look at provable security, *Proc. 31st Annu. Int. Conf. Theory Appl. Cryptographic Tech.*, vol. 7237, p. 8 [Online]. Available: <http://www.cs.bris.ac.uk/eurocrypt2012/Program/Weds/Menezes.pdf>



Ding Wang received the BS degree in information security from Nankai University, China, in 2008. Currently, he is working toward the PhD degree at Peking University, China. He has been involved in the community as a TPC member and a reviewer for several international conferences and journals. He received the Top-10 Distinguished Graduate Academic Star of the University in 2012. His research interests include cryptography and wireless network security. He is a student member of the IEEE.



Debiao He received the PhD degree in applied mathematics from School of Mathematics and Statistics, Wuhan University, China, in 2009. He is currently a lecturer at School of Mathematics and Statistics, Wuhan University. His main research interests include cryptography and information security, in particular, cryptographic protocols.



Ping Wang received the BS degree from the University of Electronic Science and Technology of China in 1983, and the PhD degree from the University of Massachusetts in 1996. Currently, he is a professor in Peking University. He served as TPC co-chairs of RFIDSec'11 Asia. He has wide interests in system security, system software, and distributed computing.



Chao-Hsien Chu is a professor of information sciences and technology at the Pennsylvania State University since 1999. His research interests mainly include information security, privacy assurance, the integration and applications of smart sensing, intelligent technologies and their applications. He has published more than 180 papers in prestigious journals and conferences. He has received five Best Paper Awards. He is a senior member of the IEEE.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.